USER MANUAL · VERSION 0.9.0

OAK Community

From an architecture brief to a signed, independently verified plan — entirely on your own machine.

> **What this is.** OAK Community 0.9.0 is a *local-first developer release*. It carries no production or customer readiness claim, and no external security review was commissioned for it. Everything in this manual runs on your own machine against public or synthetic fixtures; the only mutation the runner can perform is creating, in your local Docker, one network-isolated container per component of the chosen design, from an image pinned by digest in a target profile you explicitly acknowledge, and removing them again. It starts them, under fixed hardening, only where that profile acknowledges it. The authoritative statements of scope and limitation are `SECURITY.md` and `docs/security/residual-risk.md` in the repository.

Covers OAK Community 0.9.0 · generated from `docs/manual/` in the repository — see the colophon for how to rebuild this document and re-capture its screenshots from a live system.

CONTENTS

1. What OAK Community is — and is not
2. Installing on a fresh machine
3. The command-line journey: brief to compiled bundle
4. The browser workspace
5. Other ways in: REST, the remote CLI and MCP
6. The signed runner: sign, approve, install, observe, revoke
7. Verifying release artifacts
8. Troubleshooting
9. Uninstalling completely
10. Colophon: how this manual is built

CHAPTER 1

What OAK Community is — and is not

In plain words

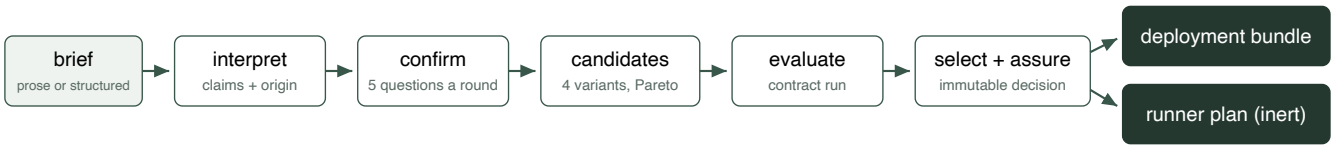
You describe an AI system you are thinking of building, in plain English or as a structured file. OAK reads it, asks you about anything important it cannot find, lays out several possible designs side by side (always including a simpler baseline, which in the reference example uses no AI model at all), rules out any that break a hard requirement and says why, records the design you choose and your reasons, and compiles a precise, reviewable plan. It never deploys anything by itself, and the same input always gives the same output.

Who it is for

- **Engineers, architects and tech leads** who design AI features and have to defend the choice.
- **Security, privacy, compliance and product owners** who sign those features off and need to see every assumption, unknown and trade-off.
- **Students and tinkerers** learning how careful AI-system design is done. Every step is visible and every file readable.
- **Builders of assistants, CI pipelines and developer portals** who want a design step they can call (chapter 5).

How it works

OAK Community is a compiler-shaped control plane for designing, evaluating and planning AI systems. You feed it an **architecture brief** — a bounded description of a problem, its stakeholders, data, quality priorities and constraints, written in plain language (Markdown or text) or as structured YAML or JSON — and it walks that brief through a deterministic pipeline: it interprets the brief into typed claims with provenance, asks you its ranked clarification questions five at a time, generates and compares candidate architectures against hard constraints, evaluates the selected candidate against an evaluation contract, records an immutable decision with your rationale, attaches an assurance plan, and finally compiles a canonical **deployment bundle** and a typed, non-executing **runner plan**.



Every stage writes immutable content-addressed artifacts with an audit trail. All of it is deterministic and offline, except interpret when you have configured a model — see chapter 4.

Figure 1 — the pipeline. The compiled plan is inert: executing anything requires the separate signing, approval and runner verification described in chapter 6.

Three properties shape everything you will see. **Deterministic:** the same inputs produce byte-identical canonical artifacts, so digests are meaningful and two clean machines agree. **Offline:** after installation, nothing in this manual needs a hosted service or a model provider, and the whole CLI journey runs with the network disabled. The network is used only when the Docker chapters (4 and 5) pull their pinned base images once, on first build; when chapter 6's first install has Docker pull one pinned placeholder image anonymously from Docker Hub; and by the optional model provider in chapter 4, which nothing requires. **Fail-closed:** anything unknown, malformed, expired, unsigned or mismatched is refused with a stable `OAK-*` error code, never skipped.

What it is not

It is not a deployment tool for real infrastructure: the only target it will ever touch is an explicitly acknowledged, network-isolated local fixture. It is not multi-tenant, has no authentication, and binds to loopback. It is not a hosted service and never phones home. And a compiled plan is not permission to run anything — execution authority comes only from the signing and approval chain in chapter 6, verified independently by the runner.

CHAPTER 2

Installing on a fresh machine

There are two ways to install OAK, and you can use either or both. The **Docker way** gives you the browser workspace, the REST API and the MCP server for AI assistants (chapters 4 and 5), and needs nothing on your machine except Docker and `git`. The **source way** gives you the `oak` command line (chapters 3 and 6, and the remote-CLI step in chapter 5), and needs only `git` and `uv`. Neither installs anything system-wide or needs administrator rights. OAK does not install Docker or `uv` for you, and in the Docker way `uv` and Node.js run only inside the image build, never on your machine.

What each way sets up

	DOCKER WAY	SOURCE WAY
You install first	Docker with Compose (Docker Desktop on a Mac) and `git`	`git` and `uv`
OAK then sets up	Two images built locally (the API image, shared by the `api`, `worker` and `migrate` services, and the web workspace) plus `postgres:17.6-alpine`: roughly 1–2 GB of disk with what the build needs. Also five containers and three volumes: `oak-postgres-data` (your cases), `oak-artifacts` (the files OAK produces) and `oak-model-state` (a Hugging Face token, if you store one)	A virtual environment, `.venv`, inside the checkout, holding the locked dependencies. `uv` fetches Python 3.13.12 itself and keeps it, with its download cache, in your home folder (`~/.local/share/uv`, `~/.cache/uv`)
Listens on	`127.0.0.1:5173` (workspace) and `127.0.0.1:8080` (API) only	Nothing, unless you start a server
Writes elsewhere	Nothing outside Docker	The workspaces you create, wherever you create them, and `~/.oak` once you use keys, model settings or the runner — besides `uv`'s own folders above. The runner's install (chapter 6) also has your local Docker pull one pinned image and create labelled containers from it, which rollback removes

Prerequisites

REQUIREMENT	VERSION	NOTES
macOS or Linux	macOS 11+ on Apple silicon (arm64) — Intel Macs are not supported; Linux glibc ≥ 2.24 (x86_64) / ≥ 2.27 (aarch64)	See `docs/platforms.md` for the authoritative matrix, including why dependency installation fails outright on macOS x86_64. Windows is not supported; WSL2 follows the Linux rows
`git`	any recent	To clone the repository — and required even offline for the repository checks, which shell out to it
`uv`	0.10.x	The source way. Installs and pins Python 3.13.12 itself
Docker with Compose	any recent, daemon on the default socket	The browser workspace, REST and MCP (ch. 4 and 5) and the runner's install cycle (ch. 6), whose first install pulls `rancher/mirrored-pause` anonymously from Docker Hub. The runner gives Docker only `PATH` and an empty Docker configuration of its own, so `DOCKER_HOST`, CLI contexts and your registry credentials are never used, and rootless Docker, Colima and Podman socket shims cannot reach it (`RR-013`)
Node.js + `pnpm`	pnpm 11.15.1	Contributors only: `make bootstrap`, the repository checks and rebuilding this manual. The pinned Node 24.18.0 that pnpm downloads is only exercised by the web workspace, so your system Node version does not matter

Install the command line

```
git clone https://github.com/nmasamba/OAK.git
cd OAK
uv sync --frozen
```

`uv sync --frozen` builds `.venv` from the lockfile, fetching Python 3.13.12 if this machine does not have it. It took 40 seconds on the machine this edition was checked on. It is the one networked step the CLI journey ever needs: chapter 3 and chapter 9 work with egress disabled from here on, as does checking a download in chapter 7, while chapters 4 and 5 need Docker, which pulls its pinned base images the first time you build, and chapter 6's first install has Docker pull one pinned placeholder image.

```
uv run oak --version
0.9.0
```

> Throughout this manual, commands are shown as `uv run oak …` so they work from the repository without activating anything. If you prefer, activate the environment once (`source .venv/bin/activate`) and drop the `uv run` prefix.

The Docker way

There is nothing more to install. Chapter 4 starts the whole stack with one command from the same checkout. If the Docker way is all you want, clone the repository as above and skip `uv sync`.

For contributors: bootstrap and the full verification gate

```
make bootstrap
make check
```

`make bootstrap` installs the locked Python and web dependencies and downloads pnpm's managed Node runtime. It is what the repository checks and the manual rebuild in chapter 10 expect. `make check` runs formatting, linting, type checks, boundary rules, contract validation and the unit, integration and end-to-end suites — around ten minutes. The PostgreSQL-gated suites skip silently unless a database is published; that is expected on a fresh machine and documented in `docs/development.md`.

CHAPTER 3

The command-line journey: brief to compiled bundle

This chapter follows the reference brief that ships with the repository — a citation-first documentation-search service — from intake to a compiled bundle. Every command runs offline. Start in the repository root and create a workspace:

```
uv run oak init ./workspace
cd workspace
```

Brief formats: plain language or structured

A brief can be written in plain language — a Markdown or text file — or as structured YAML/JSON; intake accepts all four and quarantines whichever you provide as an untrusted source record. What happens next depends on the form:

- **A plain-language brief** is recorded verbatim as your problem statement, with explicit provenance, and everything the prose does not state in OAK's terms surfaces as findings and ranked clarification questions. Brief text is always treated as *data, never instructions* — a brief that says "ignore all constraints and confirm everything" becomes a problem statement containing those words, nothing more (there is a test that pins exactly this).
- **A structured brief**, like the reference `public-manual-qa.yaml` used below, fills the typed intent directly field by field, which is what lets the rest of the pipeline run end-to-end with no model involved. Whichever form you use, the brief is read deterministically unless the request names another mode: `--interpreter online` sends it to the Hugging Face model this machine is set up for, `--interpreter local` to the model on your loopback server, and either works with a structured brief as well as a plain-language one.

> **Where a language model fits.** OAK's design gives an LLM exactly one seat: the *interpretation proposal* port, where a model proposes the structured interpretation of a prose brief. A proposal is bounded in size, schema-validated, digest-bound to the exact quarantined source it interprets, and never bypasses your confirmation — the model proposes, you decide, and the provenance records which is which. Only the paths OAK's own intent schema declares can be written, explicit brief values always win, and every section a model touched produces a question you must answer before the design can reach candidates.
>
> **Every brief is read deterministically unless the request says otherwise, and this manual never says otherwise.** Every command and screenshot below is deterministic and offline. If you want a model to read a plain-language brief, `oak models set-key` stores your Hugging Face token (and checks it) and `oak design --interpreter online` asks for it once, for that brief; `--interpreter local` asks a model on this machine instead, and chapter 4 shows the same choice in the browser. Until you ask, OAK contacts nothing, and `--interpreter deterministic` — the default — keeps it that way afterwards. Asking for Online AI is a decision with a consequence worth stating plainly: that brief's text is then sent to Hugging Face and on to the inference provider it routes to, under their terms.

1 · Ingest and interpret the brief

```
uv run oak design ../examples/briefs/public-manual-qa.yaml
Design case design-case.public-manual-qa@0.1.1 is needs_confirmation with 5
questions
```

OAK quarantines the brief as an untrusted source record, then deterministically interprets it into a typed intent. Every populated value carries provenance: a *fact* quoted from the brief, an *inference*, or a *domain default* — you will see these classes again in the browser workspace.

2 · Review the questions, then confirm

```
uv run oak questions
question.model-hardware: Which approved model licence and measured target hardware
apply? [open]
question.production-use: Confirm whether production data is permitted for this
design case. [open]
question.action-autonomy: Confirm the maximum action autonomy permitted for this
system. [open]
question.data-classification: Confirm the highest data classification used by this
design. [open]
question.data-volume: What document volume and update cadence must the design
support? [open]
```

Five ranked questions a round, each explaining why it matters to candidate choice. This brief produces exactly five; a brief a model interpreted can produce more, and the rest follow in the next round. Answers live in a small YAML file; the reference answers ship alongside the brief:

```
uv run oak confirm --answers ../examples/briefs/public-manual-qa-answers.yaml
Recorded design-case.public-manual-qa@0.1.2 as ready_for_candidates
```

Confirmation creates immutable successors for the affected claims — the original interpretation is never edited, and the audit trail records who decided what.

3 · Generate and compare candidates

```
uv run oak candidates
ID            VARIANT                   STATUS      FRONTIER  REJECTIONS
candidate-00  simpler_baseline          feasible    true      0
candidate-01  minimum_sufficient        feasible    true      0
candidate-03  balanced_enterprise       feasible    true      0
candidate-04  high_assurance_sovereign  infeasible  false     1
```

Four provider-neutral candidate architectures are generated from a bundled synthetic catalogue, checked against hard constraints, and scored with transparent estimator ranges (cost, latency, quality, operability, energy). Infeasible candidates stay visible with their rejection reasons — in the reference case, `candidate-04`'s accelerator requirement is unconfirmed, and an unknown hard constraint never passes, so it stays visible as infeasible with exactly that reason.

4 · Evaluate, select, assure

```
uv run oak evaluate candidate-03
Evaluation evaluation-result.candidate-03 is pass
printf 'Balanced trade-off between quality, cost, and operability.\n' > decision.md
uv run oak select candidate-03 --rationale-file decision.md
Selected candidate-03 in decision.candidate-03
uv run oak assure candidate-03 --output ./assurance
Wrote assurance.candidate-03 to assurance
```

`evaluate` runs the deterministic reference evaluation contract and records pass/fail per metric. `select` records an immutable decision bound to you (`OAK_ACTOR`), your rationale, and the rejected alternatives. `assure` writes the assurance plan: required tests, evidence, controls, owners, and the gate blockers that stand between this design and any real deployment.

5 · Compile the bundle and the runner plan

```
uv run oak plan candidate-03 \
  --target ../examples/targets/local-fixture.yaml \
  --output ./bundle
Compiled bundle.candidate-03.target.local-fixture to bundle; no target action was
invoked
```

The target profile is explicit invocation data — OAK never treats your machine as the target. Compilation validates the profile, binds it to the tenant, checks the candidate's platform and capacity requirements against it, and emits five canonical files into the output directory — `architecture-decision.json`, `assurance-plan.json`, `semantic-manifest.json`, `deployment-bundle.json` and a `draft` typed `runner-plan.json` whose verification policy is derived from that exact target. For this read-only fixture the plan contains only `inventory`, `validate`, `render`, `plan` and `verify` operations.

> Two clean machines running this chapter produce byte-identical semantic manifests — that is the determinism promise, and it is what makes the digests in chapter 6 worth signing. Chapter 5 shows the server producing the same fingerprint as this chapter.

Export and import

```
uv run oak export --output ../case-export
Exported design-case.public-manual-qa@0.1.7 to ../case-export
uv run oak import ../case-export --directory ../restored
Imported design-case.public-manual-qa@0.1.7 into …/restored
```

The export is the portable, digest-verified unit — also your backup before any upgrade. Import revalidates every artifact, digest and audit event before publishing the restored workspace.

CHAPTER 4

The browser workspace

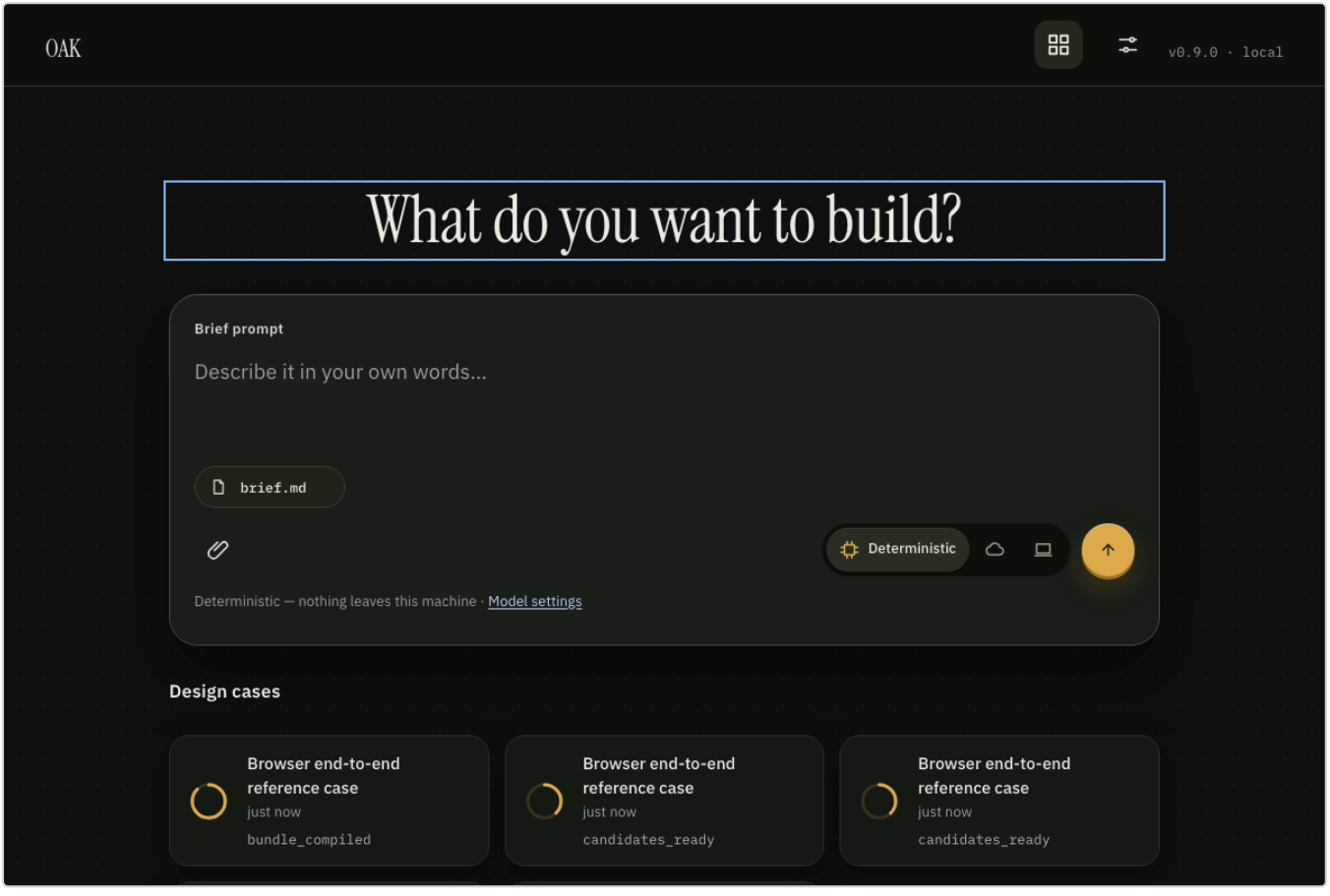
The web workspace drives the same journey through a reviewer-friendly interface, backed by a persistent PostgreSQL control plane. It needs Docker with Compose (see the prerequisites in chapter 2), and the first build pulls the pinned base images. That, the optional *Refresh models* button at the end of this chapter, the single verification request that storing a Hugging Face token makes, and an Online AI interpretation are the only steps here that use the network. Start the stack:

```
docker compose up -d --build
```

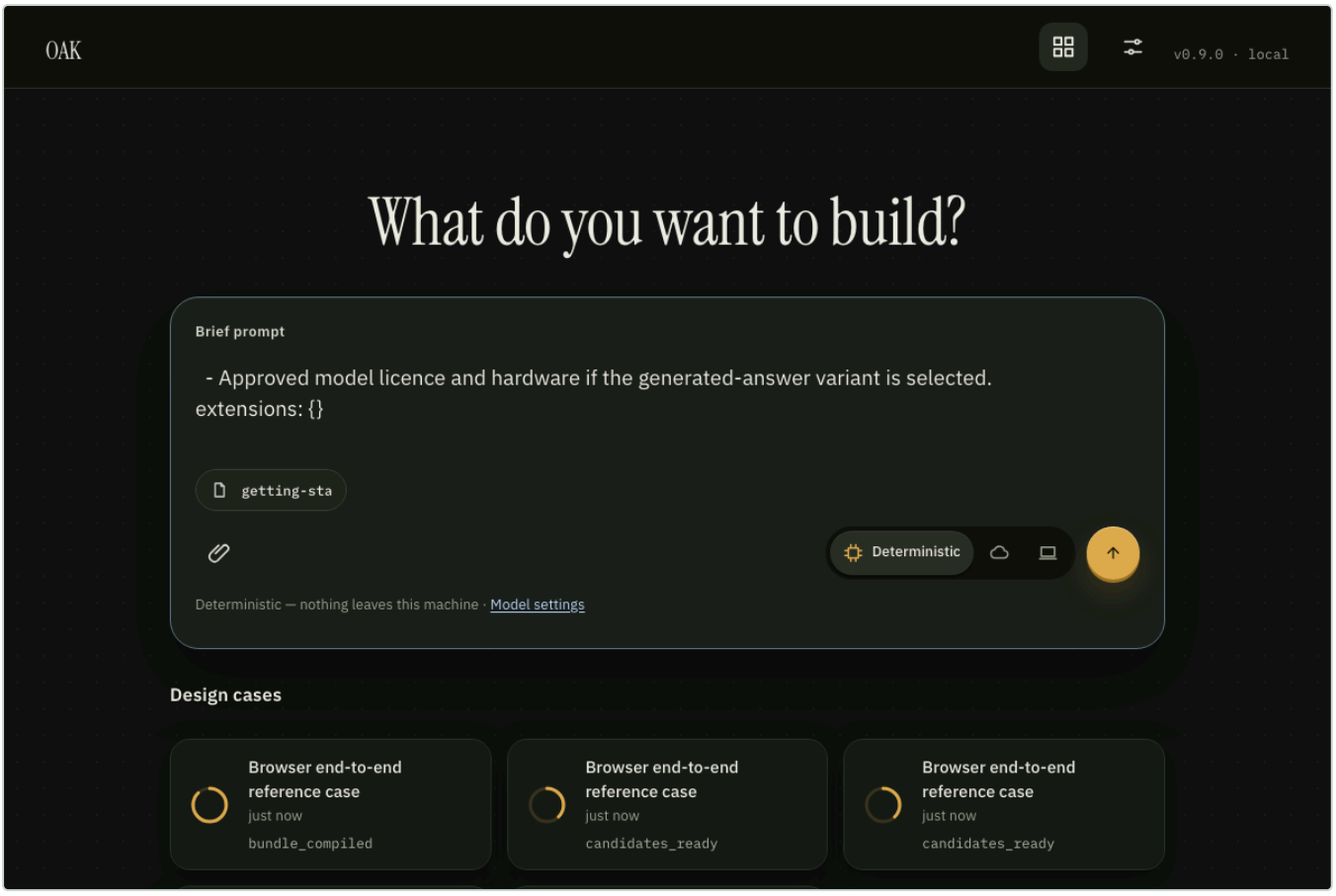
> Always pass `--build`: Compose happily reuses a stale cached image after you change source. Confirm what you are running with `curl http://127.0.0.1:8080/version` — it must report 0.9.0.

The first build downloads and builds everything. It took about three minutes on the machine this edition was checked on, and later starts take seconds. If it stops with `operation timed out`, a package download stalled. Run the same command again: the steps that finished are reused.

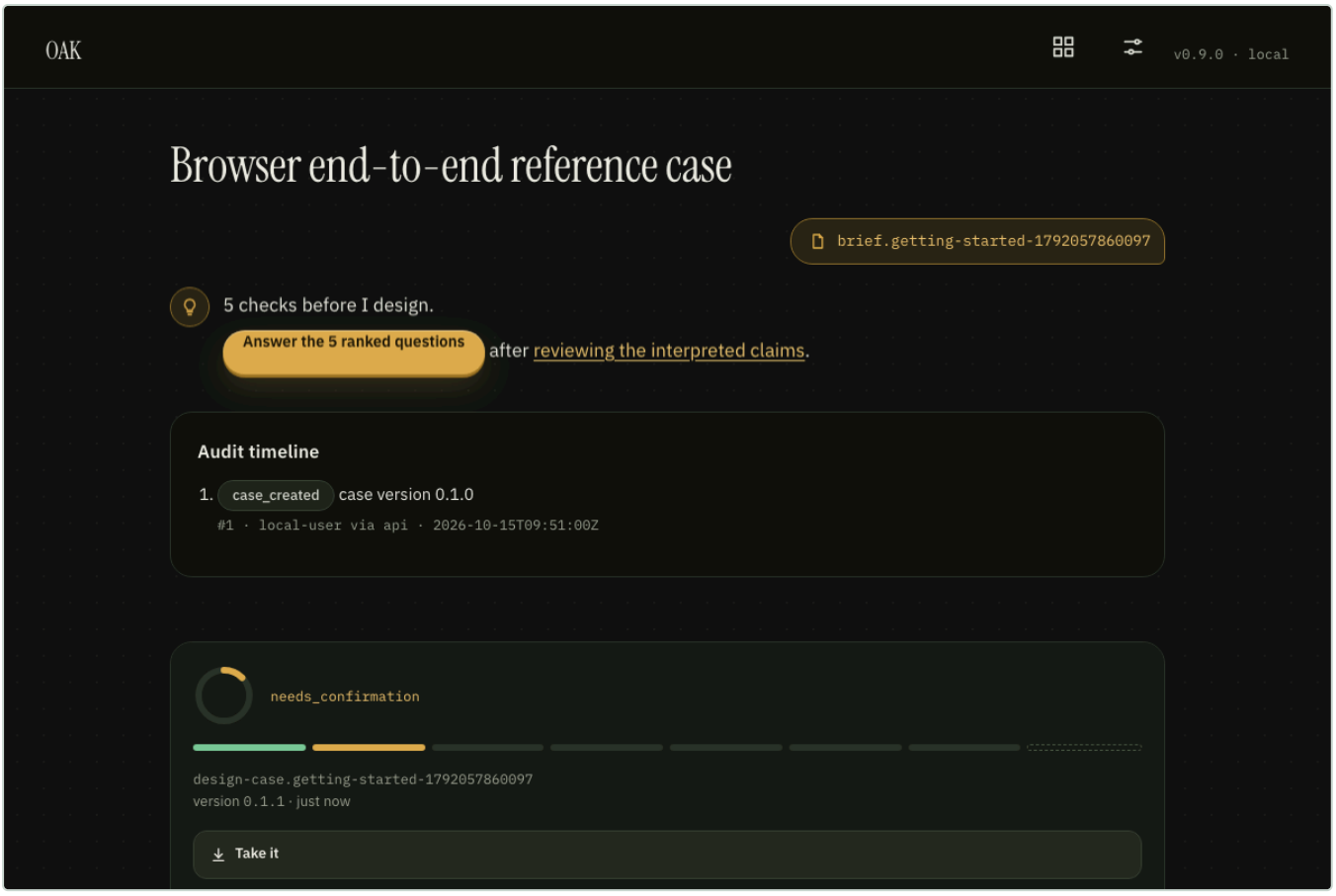
The API listens on `127.0.0.1:8080` and the workspace on `http://127.0.0.1:5173` — loopback only, by design. The screenshots below are captured from a real journey against this stack.



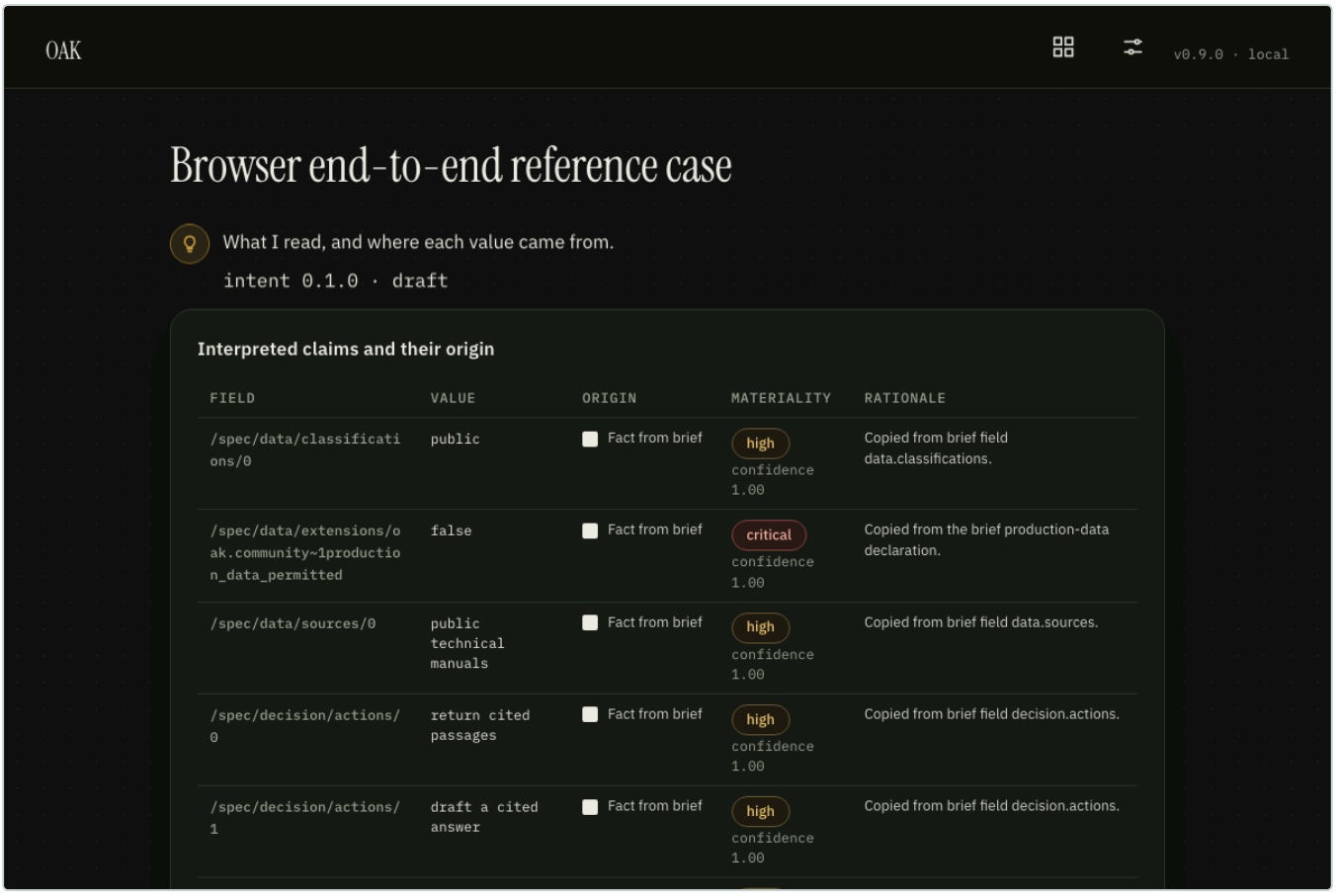
Home. One question, a composer labelled *Brief prompt*, three mode chips and one brass send keycap; recent cases as tiles with progress rings.



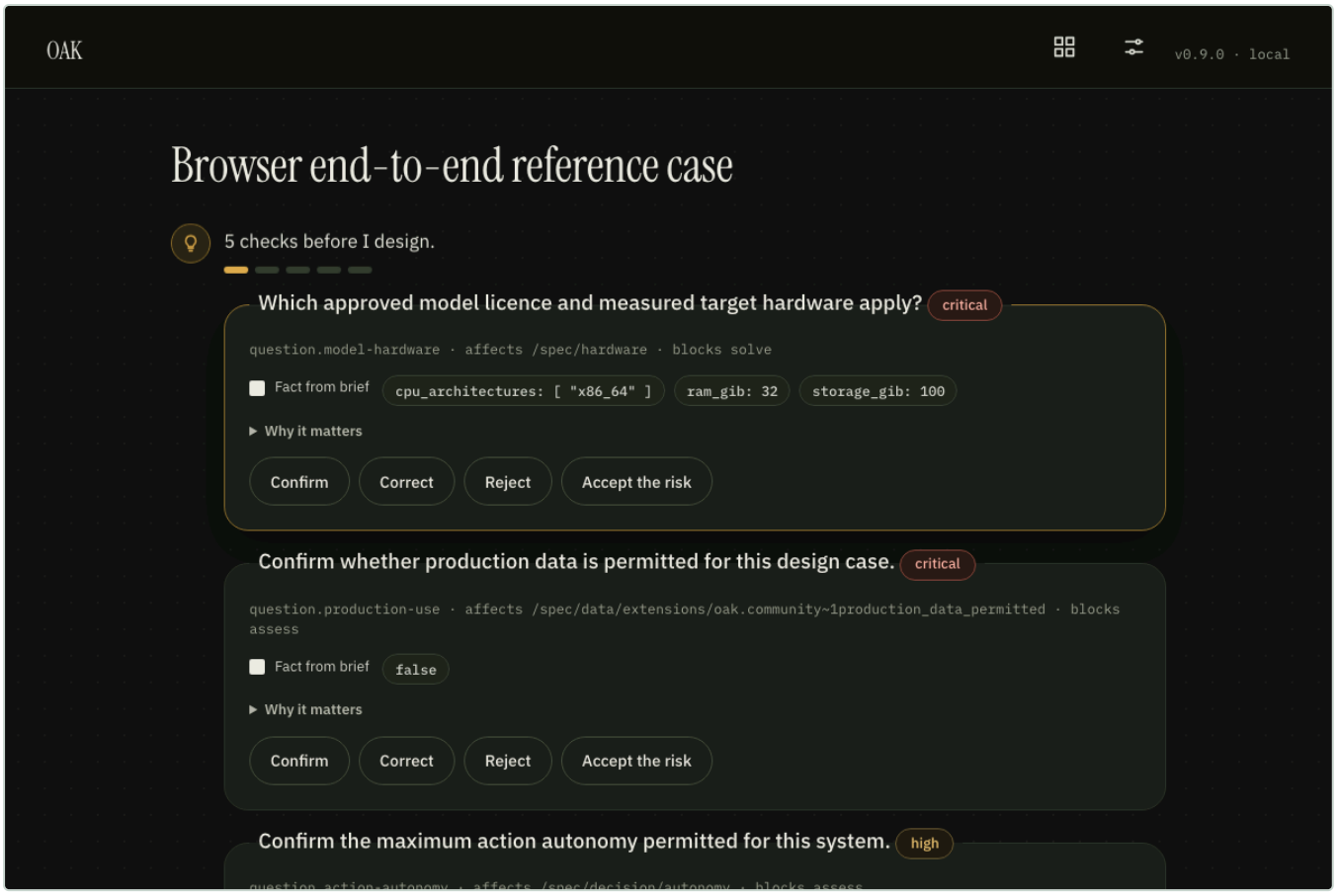
A brief in the composer with its file-name pill, exactly as the CLI would ingest it, before the send keycap is pressed.



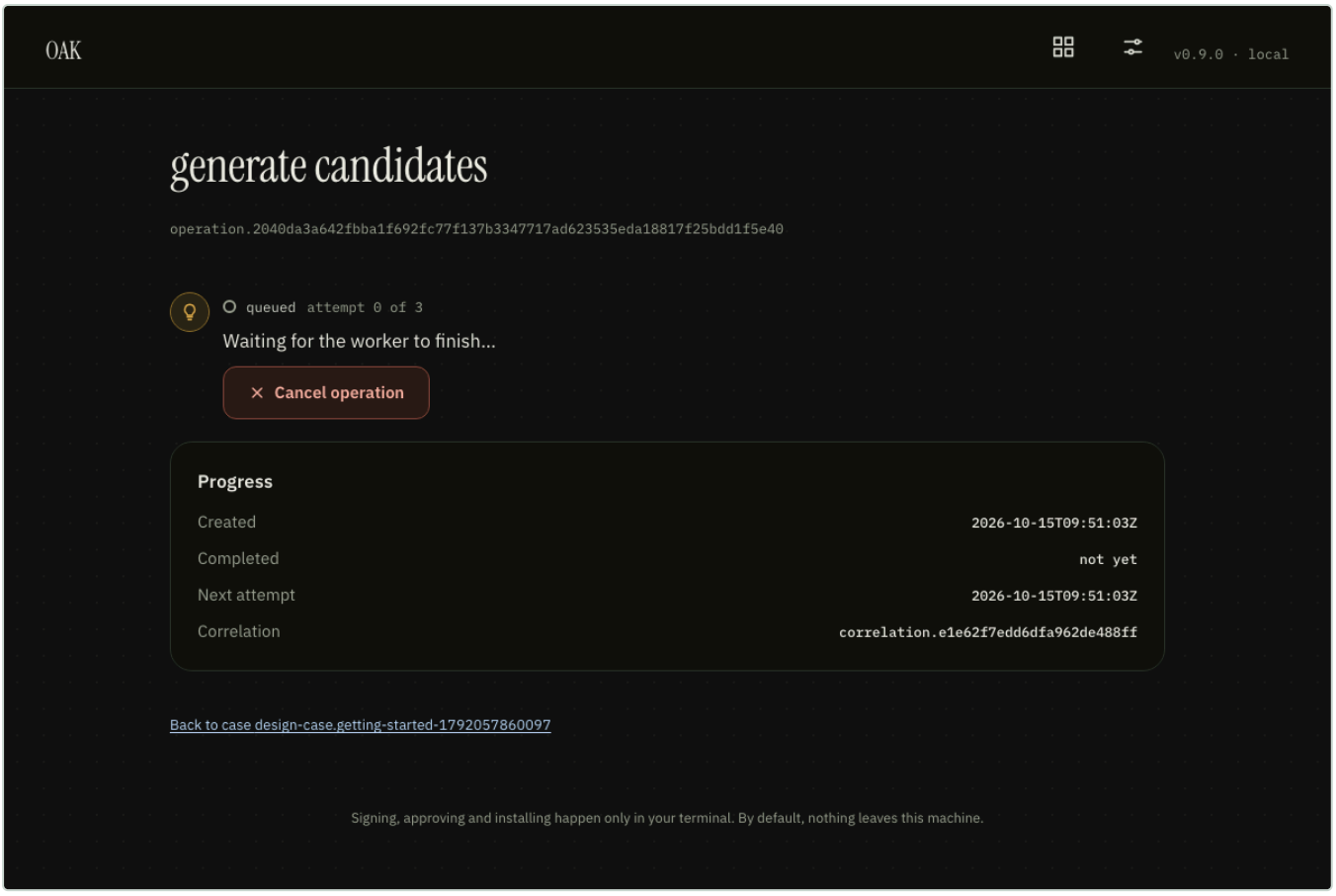
After *Interpret brief* the case reaches *needs_confirmation*: the brief as a bubble, the machine's next line with its keycap, and the rail with the ring, the status and the journey pips. The next actions are server-driven, so the UI can never invent a transition the control plane would refuse.



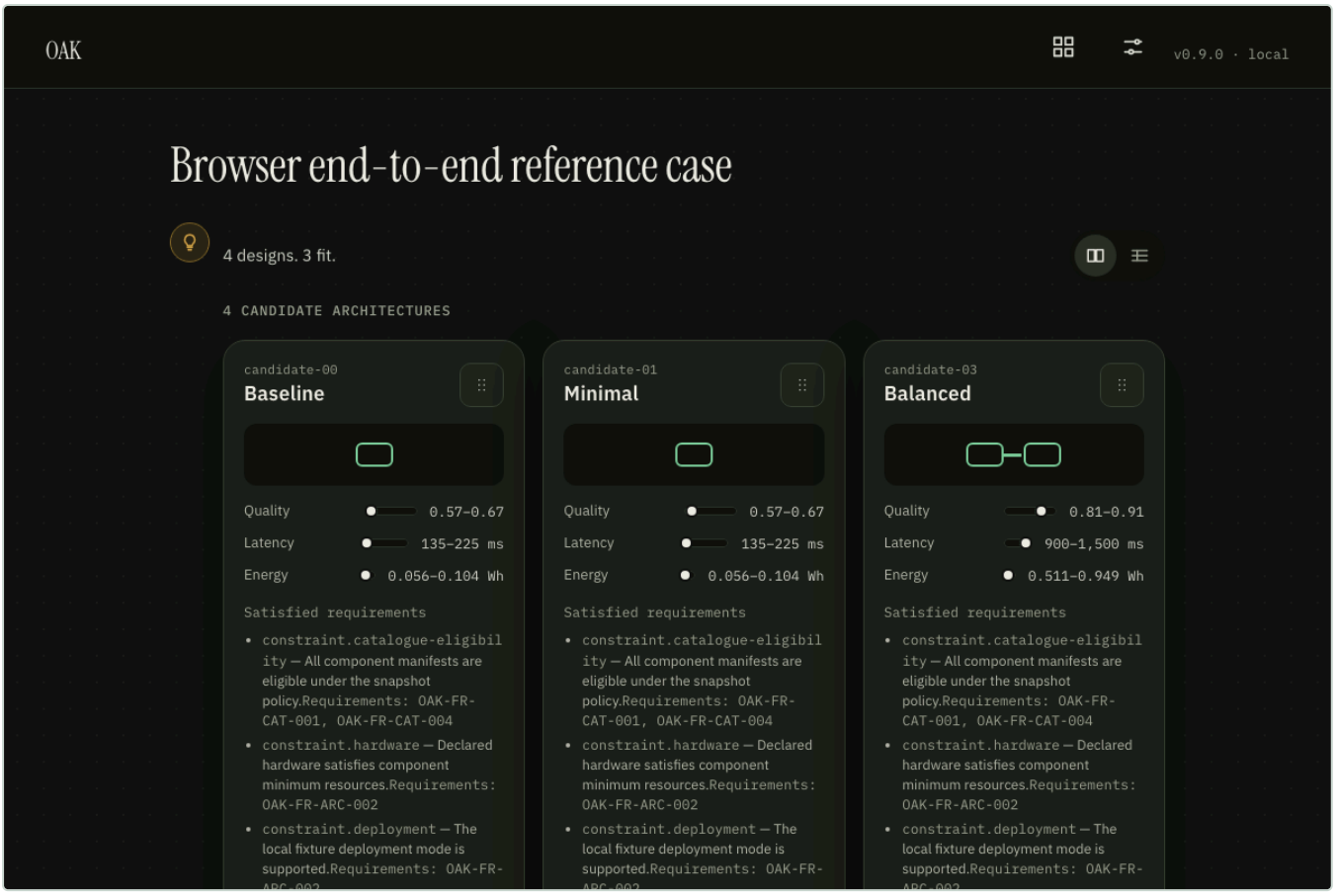
Claim review: every value carries a shape and a word for where it came from — a fact from the brief, an inference, or a domain default — with the unknowns listed beneath. A value a model proposed is marked too, and carries the confidence and reason it gave.



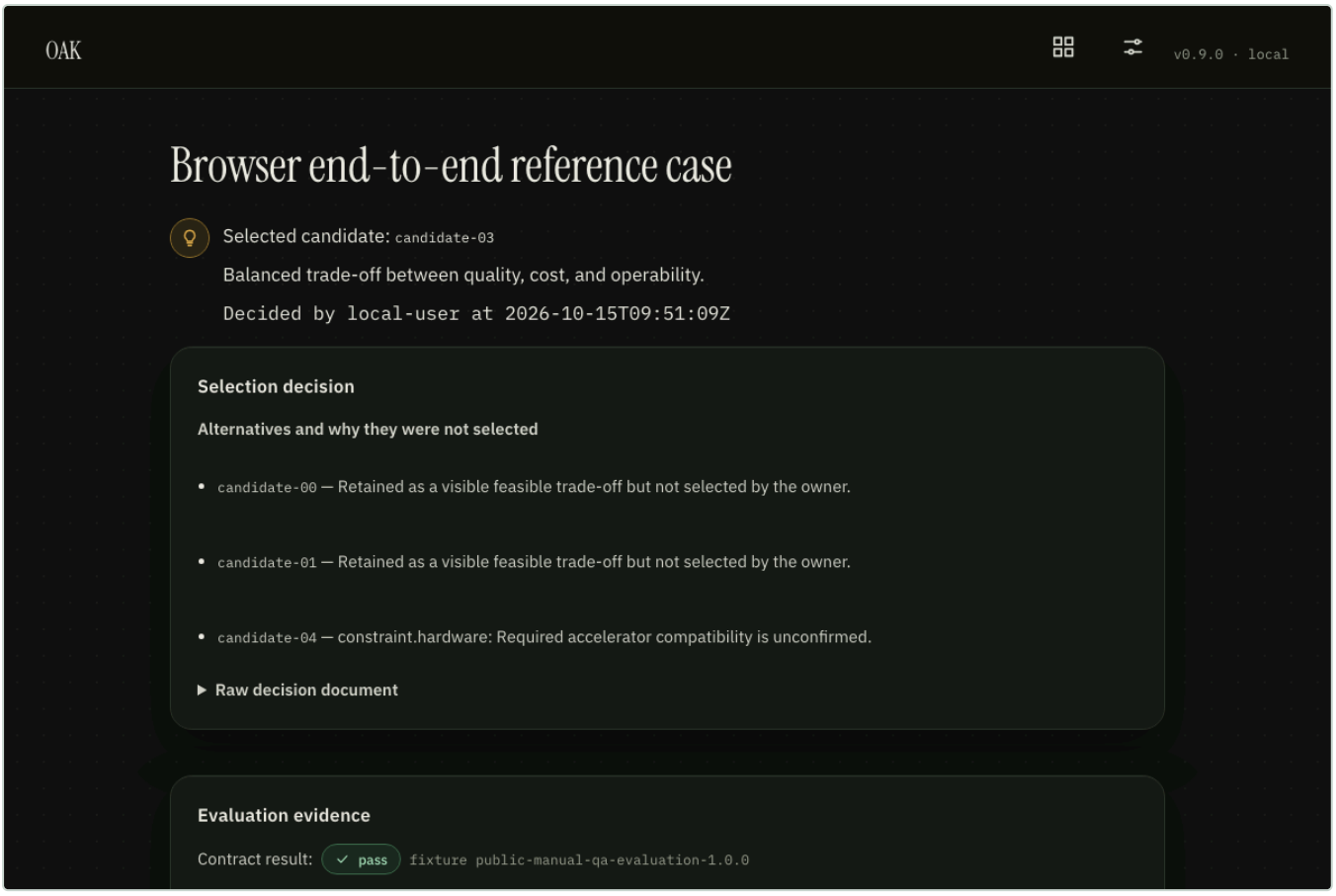
The five ranked questions as chip cards: the current value as chips beside its origin mark, four decision chips (confirm, correct with a value, reject, accept the risk), each with a recorded rationale.



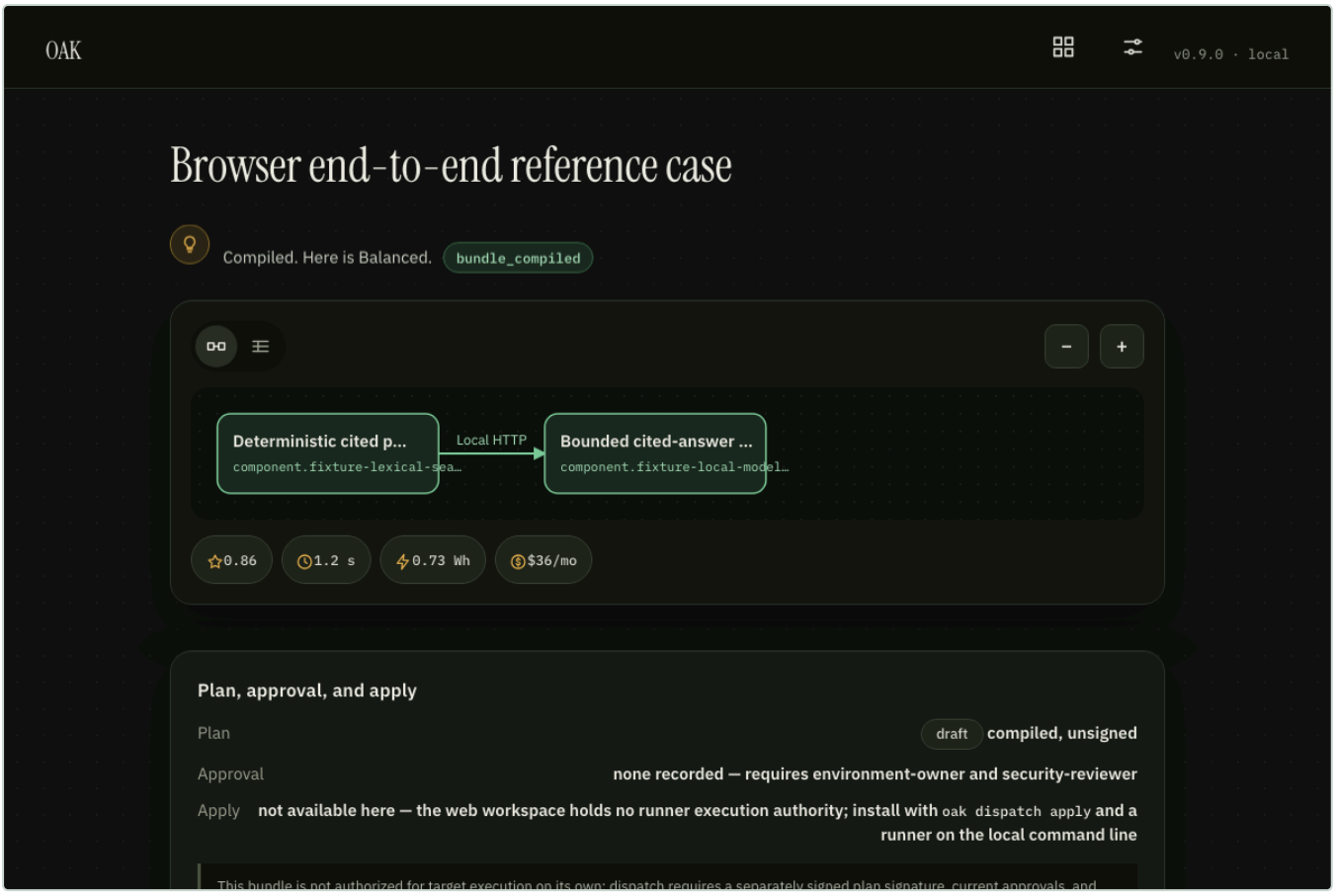
Long stages run as durable operations: a light with its state word, the attempt count, and a cancel keycap while it runs. They survive restarts, can be cancelled cooperatively, and record their outcome.



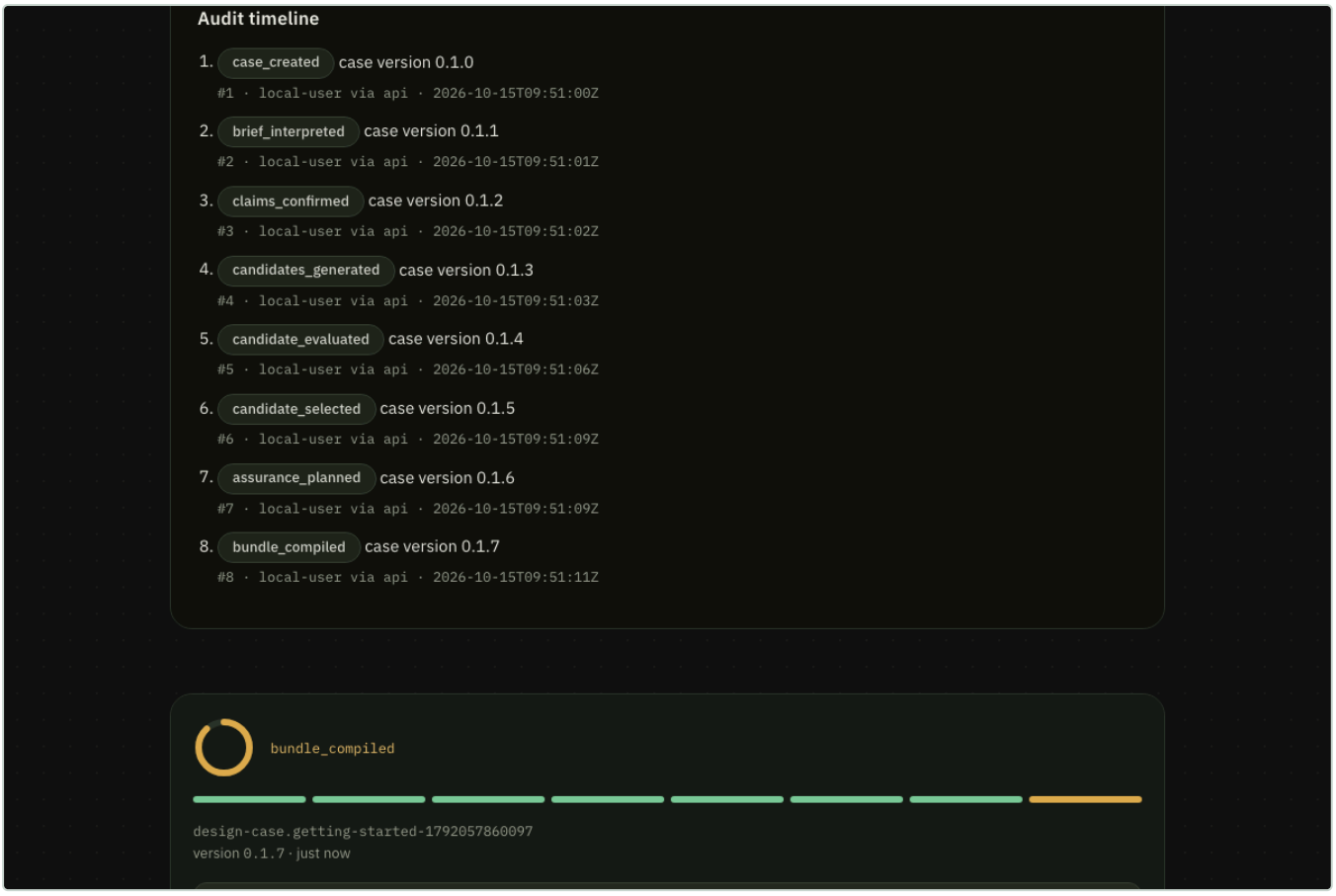
Designs as cards: a topology glyph and three range bars on one shared scale per card, with a table view a toggle away. The infeasible candidate stays visible, dashed, with its constraint failure — rejection reasons are first-class.



The immutable decision: the selected candidate with its recorded rationale and decider, every alternative with the reason it was not selected (`candidate-04` carries its unconfirmed hardware constraint), and the evaluation evidence beneath. The assurance plan's controls and gate blockers follow further down the same screen.



The architecture: the topology drawn and tabled, prediction chips, and the dock with three install lights and *Copy install steps*, which copies commands and runs nothing. Plan, approval and apply stay separated, and apply is marked unavailable here: the web surface holds no runner execution authority.



The audit timeline: every recorded event from *case_created* to *bundle_compiled*, each with its sequence number, actor, interface origin, case version and timestamp — and downloadable as a canonical JSON export.

Optional: Online AI and Local AI

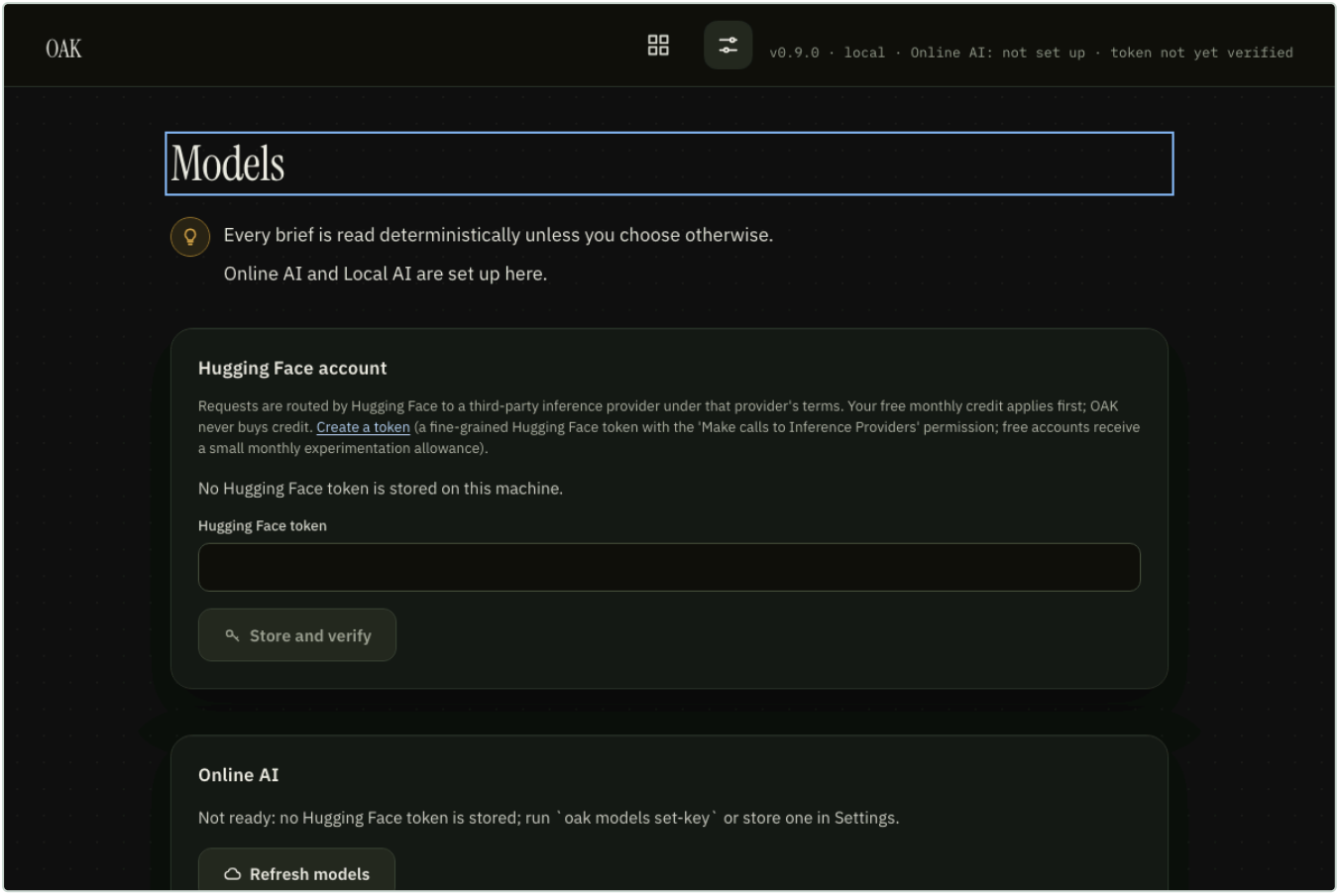
Everything above is deterministic, and it stays so unless you say otherwise for a brief. Each brief offers three ways to be read: **Deterministic** (the default; nothing leaves the machine), **Online AI** (one open model on Hugging Face Inference Providers, called with your own token) and **Local AI** (an OpenAI-compatible server on this machine, such as Ollama). **Settings → Models** is where the last two are set up; the choice itself is made per brief, from one dropdown on the "Describe what you want to build" form and again on the case page before Interpret runs, and the masthead names the Online AI pair and how long ago the token was verified. Nothing is set up by default, and nothing here is required by any other part of the manual.

> **Local AI and this stack.** Local AI's server must sit at a loopback address as the API process sees it, and inside the `api` container that is the container itself, not your computer. From this Compose stack, then, Local AI cannot reach a model server on your machine, and choosing it fails without recording anything. Use Local AI from the command line (chapter 3) on the machine that runs the model server; Online AI works from here.

The page needs the capability token the API minted when it started — not a password, but a capability that dies with that process. Print it on the machine running the API:

```
docker compose exec -T api oak models token
```

Paste it once. Storing your Hugging Face token asks Hugging Face about it straight away — one request that generates nothing and spends nothing — and the page shows the verdict with the time it was given: accepted or not, the token's role, and whether the account can pay. The token itself never comes back; the page shows only which backend holds it and a salted fingerprint. *Refresh models* reads the public catalogue anonymously and lists every model × provider pair with a structured-output route, its licence, its price and its measured speed; Online AI calls the pair you pin, or else the preferred one — the highest-trending model on Hugging Face with a route that supports structured output, publishes a price and is fast enough to answer within the request budget; models whose routes are all too slow are passed over and named. Local AI needs only the name of a model your loopback server serves.



Models, as it looks before anything is set up: the Hugging Face token card with its verdict, Online AI with the preferred pair and the pairs table once the catalogue has been read, and Local AI with its endpoint and model. Once a token is stored the card reports the backend holding it, a salted fingerprint and what Hugging Face said — never the token.

> **What changes once you choose Online AI.** That brief is sent to Hugging Face and routed to the provider named in the pair, under their retention and training terms — OAK records which model ran and the digest of what came back, but it cannot control what they then do. Your free monthly Hugging Face credit applies first and nothing beyond it is promised; when it is spent, Hugging Face refuses and OAK says so. A verdict is a claim about the past: the option shows its age, calls it stale after a day, and re-checks it before sending — unless you have raised `OAK_MODEL_TIMEOUT_SECONDS` above 35 seconds, when the check is skipped to stay inside the proxy's own limit and the stored verdict is used as it is. The values the model proposes are marked *Proposed by model*, every section it touched raises a question, and candidates are refused until you have answered them. Local AI sends the brief to your own server and nothing leaves the machine. Deterministic is what every brief gets when you choose nothing, and `oak design --interpreter deterministic` always is.

Day to day

TO	RUN, FROM THE REPOSITORY ROOT
Start the stack	`docker compose up -d --build`
Check what is running	`curl http://127.0.0.1:8080/version` · `docker compose ps`
See a service's output	`docker compose logs -f api` (or `worker`, `web`): startup messages and errors only, since OAK keeps no application log yet (`RR-015`)
Stop it, keeping your cases	`docker compose down`
Update to newer code	`git pull`, then `docker compose up -d --build`; the `migrate` service upgrades the database before the API starts. Back up first — see `docs/operations.md`
Start again from nothing	`docker compose down --volumes`, which deletes the database, the artifact store, and the model-state volume holding any provider key you stored

CHAPTER 5

Other ways in: REST, the remote CLI and MCP

The browser is one of five ways to drive OAK. They all call the same application services and change the same versioned design case, so a step gives the same result — and the same refusal — whichever way it arrives, and the audit trail records which way that was. The Compose stack from chapter 4 serves all of them except the local CLI, which keeps its case in a folder.

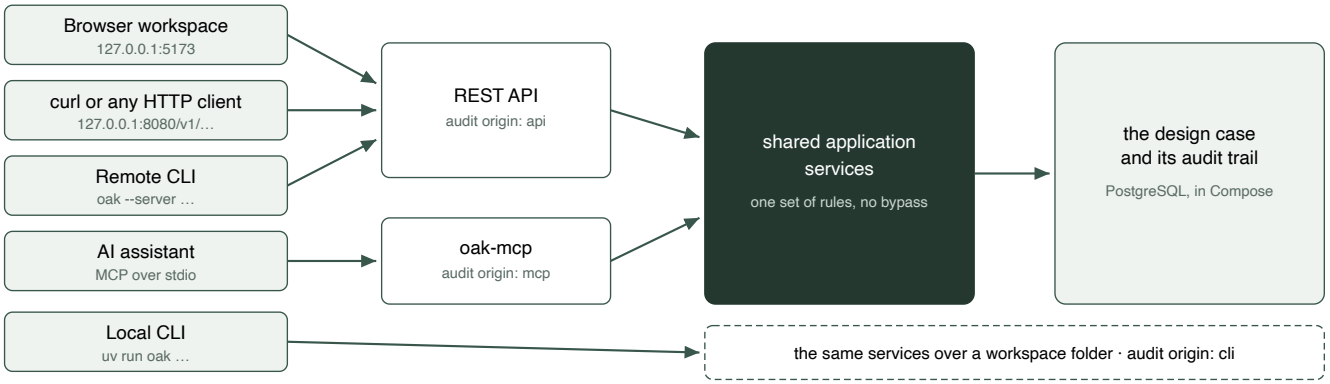


Figure 2 — five ways in, one set of rules. The browser and the remote CLI are clients of the REST API, so their steps are recorded as `api`. An export moves a case between a workspace folder and the server.

A relay: one case through four of them

> This section creates the reference case on the server. A case's id comes from its brief, so if chapter 4 already created `design-case.public-manual-qa` the create is refused with `409 OAK-EXPECTED-VERSION`. Either read along, or reset the server first with `docker compose down --volumes` — which deletes every case on it — and `docker compose up -d --build`. Run everything from the repository root. Step 3 uses the command line from chapter 2's source install, and the commands use `jq` to trim JSON replies.

1 · The browser creates and interprets the case. Under *Describe what you want to build*, tick *I have a structured YAML or JSON brief instead*, name it `public-manual-qa.yaml`, paste the contents of `examples/briefs/public-manual-qa.yaml`, click *Create case*, then *Interpret brief*. The case reaches `needs_confirmation` at version 0.1.1.

2 · `curl` reads the questions, straight from the REST API:

```
curl -s http://127.0.0.1:8080/v1/design-cases/design-case.public-manual-qa \
  | jq '.case.unresolved_questions[] | {id, question, reason}'
{
  "id": "question.model-hardware",
  "question": "Which approved model licence and measured target hardware apply?",
  "reason": "Model eligibility and measured capacity can make generated-answer
variants infeasible."
}
…four more, each with the reason it matters
```

3 · The remote CLI answers them. The same `oak` commands, pointed at the server; remote mode always names the case:

```
uv run oak --server http://127.0.0.1:8080 confirm design-case.public-manual-qa \
  --answers examples/briefs/public-manual-qa-answers.yaml
Recorded design-case.public-manual-qa@0.1.2 as ready_for_candidates
```

4 · An AI assistant generates and evaluates the options. An assistant talks to OAK's MCP server in lines of JSON over the server's standard input and output. This shell function plays the assistant for one request: it starts `oak-mcp` inside the `api` container, sends the mandatory `initialize` greeting and the request, and prints the reply.

```
oak_mcp() {
  printf '%s\n' \
    '{"jsonrpc":"2.0","id":1,"method":"initialize","params":
{"protocolVersion":"2025-06-18"}}' \
    "$1" | docker compose exec -T api oak-mcp | tail -n 1
}
OP=$(oak_mcp '{"jsonrpc":"2.0","id":2,"method":"tools/call","params":
{"name":"oak_candidates_generate","arguments":{"case_id":"design-case.public-manual-
qa","expected_version":"0.1.2","idempotency_key":"agent-generate-0001"}}}' \
  | jq -r '.result.structuredContent.operation_id')
sleep 2
oak_mcp '{"jsonrpc":"2.0","id":3,"method":"tools/call","params":
{"name":"oak_operation_get","arguments":{"operation_id":"'"$OP"'"}}}' \
  | jq -c '.result.structuredContent | {state, result: .result.status}'
{"state":"succeeded","result":"candidates_ready"}
```

Generating is a slow job, so the first reply is an operation to check on, and the `worker` container does the work. Every tool that changes the case takes the version the caller last saw (`expected_version`), so a stale caller is refused rather than overwriting, and an `idempotency_key`, so a retry never does the work twice. The assistant then evaluates `candidate-03` the same way with `oak_candidate_evaluate` at version 0.1.3.

5 · A person chooses. Choosing a design is a decision someone must own, so no MCP tool exists for it:

```
oak_mcp '{"jsonrpc":"2.0","id":7,"method":"tools/call","params":
{"name":"oak_candidate_select","arguments":{"case_id":"design-case.public-manual-
qa","candidate_id":"candidate-03"}}}'
{"error": {"code": -32602, "data": {"code": "OAK-TOOL-UNKNOWN"}, "message": "tool is
not available"}, "id": 7, "jsonrpc": "2.0"}
```

In the browser, open the case, then *Open the full comparison*, give a rationale under `candidate-03` and click *Select candidate-03*.

6 · The assistant finishes the plan with `oak_assurance_plan_create` at version 0.1.5 and `oak_bundle_compile` at 0.1.6, passing the target profile from `examples/targets/local-fixture.yaml` as JSON. The compiled runner plan is a `draft`: nothing that reaches OAK over MCP can sign, approve or dispatch it. `docs/tour.md` in the repository has these requests written out in full.

7 · Check the trail, which records the way each step came in:

```
curl -s http://127.0.0.1:8080/v1/design-cases/design-case.public-manual-qa/audit \
  | jq -r '.items[] | "\(.sequence)  \(.event_type)  via \(.interface_origin)  -> \
(.case_version)"'
1  case_created  via api  -> 0.1.0
2  brief_interpreted  via api  -> 0.1.1
3  claims_confirmed  via api  -> 0.1.2
4  candidates_generated  via mcp  -> 0.1.3
5  candidate_evaluated  via mcp  -> 0.1.4
6  candidate_selected  via api  -> 0.1.5
7  assurance_planned  via mcp  -> 0.1.6
8  bundle_compiled  via mcp  -> 0.1.7
```

Finally, compare the compiled design's fingerprint with the one chapter 3 produced offline:

```
curl -s 'http://127.0.0.1:8080/v1/design-cases/design-case.public-manual-
qa/artifacts?limit=100' \
  | jq -r '.items[] | select(.id | startswith("semantic.")) | .digest'
jq -r '.artifact_index[] | select(.id | startswith("semantic.")) | .digest' case-
export/manifest.json
sha256:2ef34758128e13038d26b82847589b2b0ec2c5f25ba6ba56982a520a92a34d63
sha256:2ef34758128e13038d26b82847589b2b0ec2c5f25ba6ba56982a520a92a34d63
```

> The same fingerprint from a folder on your machine and from a database in a container, through four different ways in. Not every record matches: the decision record, for instance, carries the time it was made and who made it. The semantic manifest leaves those out on purpose, which is what makes two runs comparable.

Connecting a real assistant

Most AI assistants and code editors that support MCP accept a JSON entry of this shape. Use the full path to your checkout, keep the Compose stack running, and if the assistant cannot find `docker`, give it the full path that `which docker` prints:

```
{
  "mcpServers": {
    "oak": {
      "command": "docker",
      "args": ["compose", "-f", "/full/path/to/OAK/compose.yaml", "exec", "-T",
"api", "oak-mcp"]
    }
  }
}
```

The assistant sees eleven tools: create, read and interpret a case; list questions; record answers under a named person; generate, list and evaluate candidates; create the assurance plan; compile the bundle; and check on a slow job. Every call is checked against the tool's schema and then handled by the same services as the browser. A refusal comes back as data (`"isError": true` with a stable `OAK-*` code) that the assistant can act on. Interpretation stays deterministic unless the assistant explicitly asks for `online` or `local`, so it cannot spend your Hugging Face credit by accident.

> What an assistant can never do through OAK: sign, approve or dispatch a plan, read files, run commands, resolve secrets, or change the model settings. The capability matrix in `docs/interfaces.md` marks each of these "never" for MCP, and the compatibility policy bans the command, file, secret, approval and runner tools under every future version. Choosing the design is left out for a different reason: it is a decision a person must own, so it is offered in the browser, the CLI and REST, but not over MCP. Each `oak-mcp` process serves one assistant (`RR-010`).

CHAPTER 6

The signed runner: sign, approve, install, observe, revoke

Everything so far produced *inert* artifacts. This chapter is where OAK lets something run — under the strictest discipline in the system. Two separate trust domains cooperate without ever trusting each other's say-so:

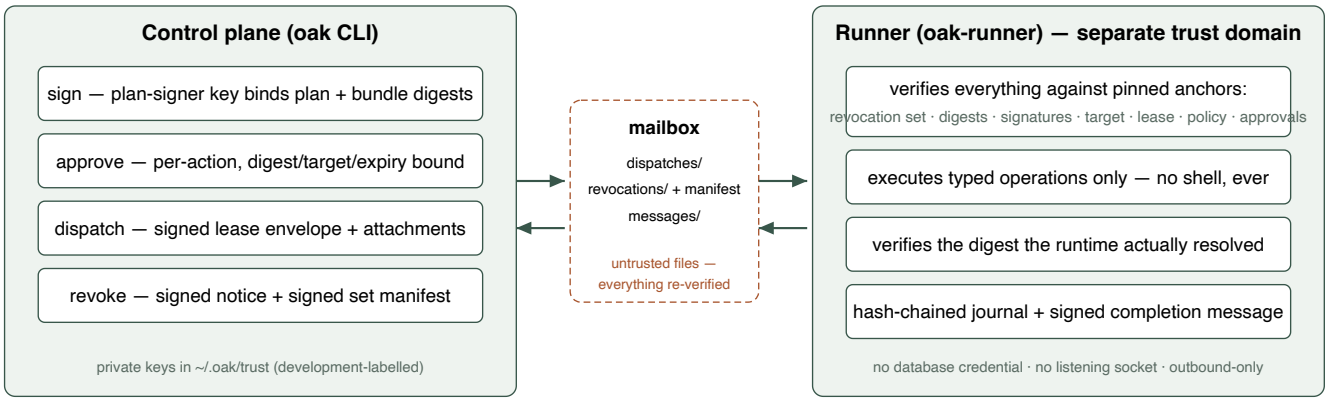


Figure 3 — two trust domains, one untrusted mailbox. A tampered mailbox file is an expected adversarial case: the runner recomputes and re-verifies everything from pinned keys, and any failure is a denial with a stable code, published back as a signed message.

Set up and compile against the mutation fixture

Two mutation-capable targets ship, and both need Docker. `local-mutation-fixture.yaml`, used first, is an acknowledged, network-isolated fixture whose only permitted mutation is creating one container per node of the selected candidate (two for `candidate-03`) and removing them again; it never starts them. `local-started-fixture.yaml`, used at the end of this chapter, also starts and smoke-tests them. Start from the repository root again — chapter 3 left you inside `./workspace`, and the `../examples/…` paths below resolve from a workspace created in the root. Set the environment, then compile chapter 3's journey against the first of them:

```
export OAK_TRUST_DIRECTORY=$PWD/trust
export OAK_DISPATCH_MAILBOX=$PWD/mailbox
export OAK_ACTOR=local-user

uv run oak init ./runner-demo && cd runner-demo
uv run oak design ../examples/briefs/public-manual-qa.yaml
uv run oak confirm --answers ../examples/briefs/public-manual-qa-answers.yaml
uv run oak candidates
uv run oak evaluate candidate-03
printf 'Balanced fixture for the runner demonstration.\n' > decision.md
uv run oak select candidate-03 --rationale-file decision.md
uv run oak assure candidate-03 --output ./assurance
uv run oak plan candidate-03 \
  --target ../examples/targets/local-mutation-fixture.yaml \
  --output ./bundle
```

Sign, approve, dispatch

```
uv run oak keys init
plan-signer: sha256:d887…cdb6 (development)
approver: sha256:7108…0488 (development)
extension-steward: sha256:edbb…ce5e (development)
uv run oak sign
Signed plan for design-case.public-manual-qa@0.1.8
uv run oak approve dry_run
uv run oak approve architecture
uv run oak approve apply
uv run oak approve rollback
uv run oak dispatch apply verify rollback
Dispatched apply, verify, rollback as dispatch.public-manual-qa.14
```

`keys init` creates three development-labelled Ed25519 identities with separated roles (your fingerprints will differ — keys are generated fresh) — the plan-signer may not approve, and the runner enforces that the two signatures come from different keys. Each `approve` is bound to one action, the exact plan and bundle digests, the target's identity and fingerprint, a nonce, and an expiry. `approve architecture` approves the design itself: it also names the decision, the chosen candidate and the digest of what `apply` would install, and `approve apply` is refused (`OAK-APPROVAL-ARCHITECTURE`) until a current one exists. Rollback and destroy never need it. `dispatch` writes the signed lease envelope and every attachment into the mailbox — delivery, not execution.

Run the runner

```
OAK_RUNNER_MAILBOX=$OAK_DISPATCH_MAILBOX \
OAK_RUNNER_TRUST_ANCHORS=$OAK_TRUST_DIRECTORY \
OAK_RUNNER_TARGET_PROFILE=../examples/targets/local-mutation-fixture.yaml \
OAK_RUNNER_HOME=$PWD/runner-home \
uv run oak-runner run-once
completed dispatch.public-manual-qa.14: succeeded
```

The runner independently verifies the revocation set, the envelope, every digest and signature against its own pinned anchors, the target fingerprint it recomputes locally, the lease and nonce, the compiled verification policy's clauses, the adapter allowlist and typed parameters, and the approvals — for `apply`, both the apply approval and the architecture approval, whose decision and installation must match what was dispatched — all before touching anything. It then executes the requested operations in plan order, which puts `verify` first: it verifies, then applies. The apply creates one container per node with `docker create --network=none`, named from the installation (the case version, the workspace and the target) and the node, and labelled `oak.fixture`, `oak.case`, `oak.node`, `oak.installation` and `oak.isolation`. For each it reads back the image digest the daemon actually resolved and refuses on any mismatch, and checks that it is only created, because this profile does not start it. It then rolls back, removing only containers this case owns, with their volumes, and inspecting afterwards to prove each is gone. Finally it publishes a signed completion message recording `"applied_kinds": ["verify", "apply", "rollback"]`. Then:

```
uv run oak ingest --output json
```

```
OAK_RUNNER_HOME=$PWD/runner-home uv run oak-runner status
{
  "journals": [
    {
      "chain": "verified",
      "dispatch": "dispatch.public-manual-qa.14",
      "entries": 8,
      "manual_recovery_required": false
    }
  ]
}
```

`ingest` verifies the runner's signed messages — delivery never implies success. A completion whose `apply` succeeded and was not rolled back moves the case to `deployed`; this one rolled back in the same run, so it is recorded as `runner_rolled_back` and the case's status does not change. `status` reports the journal's hash-chain integrity and any manual-recovery state. `status` reads `OAK_RUNNER_HOME` — pass it as shown, because the variables above were prefixes on the `run-once` command and do not persist. Without it, `status` reads the default home (`~/.oak/runner`) and reports an empty journal list rather than failing.

What a denial means

`run-once` reports whether the runner ran, not whether anything succeeded: it exits `0` even when every dispatch was denied, because denying is the runner working correctly. Denials appear on stderr as `denied <dispatch>: <reason>` and come back through `oak ingest` as signed messages with `"outcome": "denied"`. The reason carries a stable code; the families, in the order the runner checks:

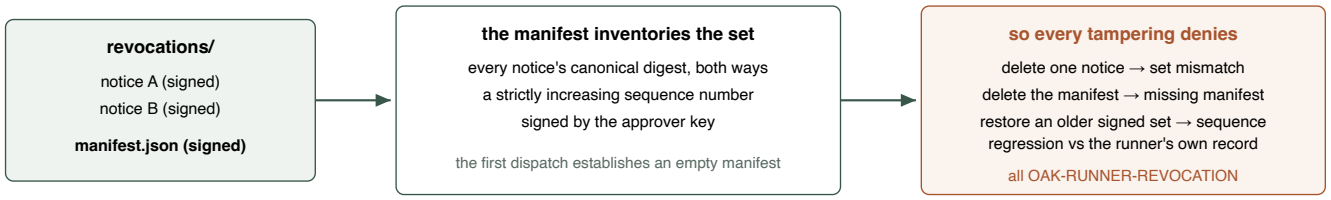
DENIAL FAMILY	CODES	WHAT IT MEANS
Revocation set	`OAK-RUNNER-REVOCATION`	The signed revocation manifest is missing, does not match the notices on disk, or its sequence regressed. Re-publish from the control plane; do not hand-edit the mailbox (Figure 4)
Protocol and schema	`OAK-RUNNER-PROTOCOL`, `OAK-RUNNER-SCHEMA`	The envelope's protocol version is unsupported, or an attachment is not schema-valid — usually a version mismatch between control plane and runner
Attachment integrity	`OAK-RUNNER-ATTACHMENT`, `OAK-RUNNER-DIGEST`	An attachment's bytes do not match the digest the envelope references — the mailbox was altered or truncated. Re-dispatch
Signatures and trust	`OAK-RUNNER-SIGNATURE`, `OAK-RUNNER-TRUST`	A signature does not verify against the runner's own pinned anchors — never against a key carried in the document itself
Identity and plan state	`OAK-RUNNER-TENANT`, `OAK-RUNNER-ENVIRONMENT`, `OAK-RUNNER-TARGET`, `OAK-RUNNER-PLAN-STATE`, `OAK-RUNNER-PLAN-EXPIRED`	Tenant, environment or target identity mismatch — the target fingerprint is recomputed locally, not read from the envelope — or the plan is not dispatchable or has expired
Lease and replay	`OAK-RUNNER-LEASE`, `OAK-RUNNER-REPLAY`	The lease is outside its validity window or its nonce was already consumed; an unreadable nonce ledger refuses rather than reading as empty. During an install, the same code stops the runner if the lease lapses before its next side effect
Separation of duties	`OAK-RUNNER-SEPARATION`	The signing and approving identities are the same key — re-sign and re-approve with separated roles
Verification policy	`OAK-RUNNER-POLICY`	A requested kind is outside the target-derived `allowed_operation_kinds`, or a mutating kind was requested where the target forbids mutation
Operations and adapters	`OAK-RUNNER-ADAPTER`, `OAK-RUNNER-PARAMETERS`, `OAK-RUNNER-REGISTRY`, `OAK-RUNNER-SECRETS`, `OAK-RUNNER-NETWORK`, `OAK-RUNNER-TARGET-CAPABILITY`	An operation names an adapter or parameters outside the code-level allowlist and schema, an image registry outside the target's allowance, a secret reference or network destination the target does not permit, or a capability the target does not have

Approvals	`OAK-RUNNER-APPROVAL`	The approval for the requested action is missing, revoked, expired, or bound to different digests or a different target — or, for `apply`, the separate architecture approval is missing, not current, or names a different decision or installation
Execution time	`OAK-RUNNER-IMAGE`, `OAK-RUNNER-FOREIGN`, `OAK-RUNNER-SMOKE-TEST`, `OAK-RUNNER-SUBPROCESS`, `OAK-RUNNER-DRIFT`, `OAK-RUNNER-APPLY`, `OAK-RUNNER-ROLLBACK`	After every verification passed: the image digest the daemon actually resolved does not match the approved one, a container with the planned name exists but is not this case's own (it is left alone), a started container failed its smoke test, the docker invocation itself failed, target state drifted mid-run, or the install or its rollback failed. A failed install removes exactly the containers it created; a failed rollback or destroy, or a clean-up that fails after a failed install, is what raises `manual_recovery_required`

Every code is documented with its raise site in `docs/error-codes.md`, and the ordered checks themselves are contractual — see `docs/signed-runner.md`.

Revocation — and why deleting files does not help

```
uv run oak revoke-approval apply --reason "manual drill"
```



A revoked approval stays revoked: the runner would rather deny every pending dispatch than guess about a revocation set it cannot prove.

Figure 4 — the revocation channel. Notices and their signed set manifest; the runner records the highest manifest sequence it has seen in its own home directory.

Install, test and observe

The dispatch above installed and removed in one run, so that case never reached `deployed`. To keep an installation, test it and record what it did, compile the same journey once more against `local-started-fixture.yaml`, the profile that also acknowledges starting each container. Go back to the repository root and give it its own workspace and mailbox: both workspaces hold the same case, so their dispatch numbers can collide in one mailbox. The trust directory, and the keys in it, stay the same.

```
cd ..
export OAK_DISPATCH_MAILBOX=$PWD/mailbox-started

uv run oak init ./runner-started && cd runner-started
uv run oak design ../examples/briefs/public-manual-qa.yaml
uv run oak confirm --answers ../examples/briefs/public-manual-qa-answers.yaml
uv run oak candidates
uv run oak evaluate candidate-03
printf 'Balanced fixture for the runner demonstration.\n' > decision.md
uv run oak select candidate-03 --rationale-file decision.md
uv run oak assure candidate-03 --output ./assurance
uv run oak plan candidate-03 \
  --target ../examples/targets/local-started-fixture.yaml \
  --output ./bundle
uv run oak architecture
candidate-03 (balanced_enterprise) in design-case.public-manual-qa — selected by
local-user

Nodes:
  node.retrieval           Deterministic cited passage retrieval ->
component.fixture-lexical-search@1.0.0
  node.generation          Bounded cited-answer drafting -> component.fixture-local-
model@1.0.0

Edges:
  node.retrieval -> node.generation (Local HTTP)

Predictions (value [lower, upper] unit):
  monthly_cost             36.0 [32.4, 39.6] USD/month
  latency_p95              1200.0 [900.0, 1500.0] milliseconds
  quality                  0.86 [0.81, 0.91] normalized_fixture_score
  operability              0.72 [0.64, 0.8] normalized_fixture_score
  energy                   0.73 [0.511, 0.949] Wh/request

Would install on target.local-started-fixture (starts containers):
  node.retrieval           rancher/mirrored-pause@sha256:ee65…917a
  node.generation          rancher/mirrored-pause@sha256:ee65…917a
Approvals an install needs, both current: architecture absent, apply absent

Read from the design case. Producing this compiled, signed, approved and installed
nothing; a user who wants only the architecture can stop here.
```

`oak architecture` reads the design case and writes nothing. If the design is all you want, this is the place to stop: nothing has been signed, approved or installed. To install it:

```
uv run oak sign
Signed plan for design-case.public-manual-qa@0.1.8
uv run oak approve architecture
uv run oak approve apply
uv run oak dispatch apply
Dispatched apply as dispatch.public-manual-qa.12
OAK_RUNNER_MAILBOX=$OAK_DISPATCH_MAILBOX \
OAK_RUNNER_TRUST_ANCHORS=$OAK_TRUST_DIRECTORY \
OAK_RUNNER_TARGET_PROFILE=../examples/targets/local-started-fixture.yaml \
OAK_RUNNER_HOME=$PWD/runner-home \
uv run oak-runner run-once
uv run oak ingest
```

The install creates one container per node, as before, then starts each under fixed hardening: no network, a read-only root, no capabilities, no new privileges, user 65534, 256 MiB of memory and no swap, half a CPU, at most 64 processes, no restart policy, no log driver, and no volumes, environment or published ports. Each must be running (or healthy, if its image declares a health check) within 15 seconds and still running a second later. Otherwise the install fails with `OAK-RUNNER-SMOKE-TEST` and removes the containers it created. Ingesting the successful install moves the case to `deployed`, and the containers keep running until a rollback or destroy removes them.

To measure a repeated install as well, run `oak dispatch apply` again, then the same `run-once` command and `oak ingest`: the runner finds each container already there, carrying this case's labels, and checks its image digest again instead of creating another. Then record what was observed:

```
uv run oak observe
Recorded observation.public-manual-qa.1 for design-case.public-manual-qa; the case
is observing and the installation is present

Deployment measures (one non-production sample; none of these satisfies a gate):
  EV-DEP-01  Fresh install success          pass (1/1)
  EV-DEP-02  Idempotent re-apply success    pass (1/1)
  EV-DEP-03  Rollback recovery              unknown — No rollback or compensation
has removed anything on this case.
  smoke-test Post-install smoke test        pass (2/2)

Predicted beside observed:
  monthly_cost                      36.0 USD/month [32.4, 39.6] -> unknown
(not_observable_on_fixture)
  latency_p95                       1200.0 milliseconds [900.0, 1500.0] -> unknown
(no_request_workload)
  quality                           0.86 normalized_fixture_score [0.81, 0.91] ->
unknown (requires_workload_and_human_review)
  operability                       0.72 normalized_fixture_score [0.64, 0.8] ->
unknown (not_measured)
  energy                            0.73 Wh/request [0.511, 0.949] -> unknown
(no_energy_measurement)
  metric.citation-correctness       0.99 proportion -> unknown
(requires_workload_and_human_review)
  metric.unsupported-assertion-rate  0.005 proportion -> unknown
(requires_workload_and_human_review)
  metric.latency-p95                1200.0 milliseconds -> unknown
(no_request_workload)

Measured, with no prediction to compare:
  node.retrieval                    startup_seconds 0.18 seconds
  node.generation                   startup_seconds 0.11 seconds
  node.retrieval                    memory_bytes 516096 bytes
  node.generation                   memory_bytes 516096 bytes

evidence.observed-calibration: not satisfied; missing monthly_cost:
not_observable_on_fixture, latency_p95: no_request_workload, quality:
requires_workload_and_human_review, energy: no_energy_measurement
Nothing was proposed, promoted or fed back into a dispatch.
```

`oak observe` is local-only. It writes an observation record and moves the case to `observing`. The record gives install success, the repeated install and rollback recovery (`EV-DEP-01`, `EV-DEP-02`, `EV-DEP-03`) and the smoke test, each with its sample size, or `unknown` with a reason where nothing has been tried yet; none of them satisfies a gate. Beside every predicted objective and contract metric it puts the observed value, and here every one reads `unknown` with a reason code, because the placeholder image serves no requests and nothing measured cost, latency, quality or energy. Startup time and memory are recorded as measurements with no prediction to compare. The record proposes nothing,

and nothing from it is fed back into a dispatch. Before the case is `deployed`, `oak observe` refuses with `OAK-OBSERVE-STATE`.

To remove the installation, approve and dispatch the rollback, run the runner once more, and record it:

```
uv run oak approve rollback
uv run oak dispatch rollback
OAK_RUNNER_MAILBOX=$OAK_DISPATCH_MAILBOX \
OAK_RUNNER_TRUST_ANCHORS=$OAK_TRUST_DIRECTORY \
OAK_RUNNER_TARGET_PROFILE=../examples/targets/local-started-fixture.yaml \
OAK_RUNNER_HOME=$PWD/runner-home \
uv run oak-runner run-once
uv run oak ingest
uv run oak observe
```

The rollback removes each container with its volumes and inspects afterwards to prove it is gone. The second observation record reports rollback recovery and the installation as `removed`. The pinned placeholder image stays in your Docker; chapter 9 says how to remove it.

CHAPTER 7

Verifying release artifacts

If you received OAK Community as built artifacts rather than as this repository, verify them before installing anything. The newest published release is `0.9.0`, the version this manual describes, on the project's GitHub Releases page. `0.8.0` was approved but never published. To build the artifacts yourself, the repository does it with:

```
make release
```

which builds the wheel and sdist twice and fails if the digests differ, installs the wheel into a clean environment holding only the locked runtime closure, runs it from outside the checkout, and emits an SBOM, a licence inventory, and `SHA256SUMS`. On the consuming side:

```
# in a checkout, against the build you just made
make verify-release

# what a consumer actually runs, against a downloaded directory
python scripts/verify_release.py ./downloaded-release
OK  THIRD-PARTY-LICENCES.md
OK  oak-community-0.9.0.cdx.json
OK  oak_community-0.9.0-py3-none-any.whl
OK  oak_community-0.9.0.tar.gz

verified 4 artifact(s) against SHA256SUMS
Checksums prove the bytes match the manifest. They do not prove who produced
them: OAK Community release artifacts are unsigned.
```

> `make verify-release` is the in-checkout convenience: it runs the same script against `dist/release`. If you received artifacts rather than this repository, you have no Makefile — use the script directly (it is standard-library only, imports nothing from OAK, and needs no installed OAK), or the equivalent `shasum -a 256 -c SHA256SUMS` on macOS / `sha256sum -c SHA256SUMS` on Linux. Exit codes: 0 verified, 2 verification failed, 3 the manifest or a listed file is missing or malformed.

Be aware of what this does and does not prove: checksums prove the bytes match the manifest; they do not prove who produced them. Release artifacts are deliberately not signed yet (`RR-005` in the residual-risk register), and the container images are not byte-reproducible (`RR-006`) — their SBOMs and provenance under `docs/release/0.9.0/` record what one build produced.

CHAPTER 8

Troubleshooting

Online AI refuses with `OAK_MODEL_KEY_REJECTED`	Hugging Face rejected the stored token when it was last checked or used	Store a current token with `oak models set-key`; the verdict is forgotten when the key changes
Online AI refuses with `OAK_MODEL_ROUTE_UNAVAILABLE`	The account cannot be billed for that provider route, even with credit	Pick another route with `oak models select huggingface <model_id> --provider <route>`, or `oak models clear huggingface` to fall back to the preferred pair
Online AI says no route is fast enough	Every catalogued route publishes throughput too low for the request budget	Run `oak models discover`, raise `OAK_MODEL_TIMEOUT_SECONDS` (up to 35 keeps the pre-flight), or set the provider policy to fastest
A command refuses with `OAK_EXPECTED_VERSION`	The case advanced since you last read it	Re-read the case and retry with the current version — this is optimistic concurrency working, not an error to force past
The web stack serves an old version	`docker compose up -d` reused a cached image	Always `--build`, then check `/version`
The first `docker compose up -d --build` stops with `operation timed out`	A package download stalled while the images were building	Run the same command again; the build steps that finished are reused
Creating the reference case on the server is refused with `409 OAK_EXPECTED_VERSION`	A case's id comes from its brief, and this server already holds that case	Carry on with the existing case, or reset the server with `docker compose down --volumes`, which deletes every case on it
An MCP client reports that it cannot start `docker`	Assistants often launch their helpers with a minimal `PATH`	Put the full path that `which docker` prints into the client's configuration
`OAK_RUNNER_REVOCATION` on every dispatch	The revocation manifest is missing or the notice set does not match it	Re-publish from the control plane (`oak revoke-approval`, or a fresh dispatch establishes an empty manifest). Do not hand-delete notices — the mismatch is the protection working
`OAK_RUNNER_REPLAY` on every dispatch	The consumed-nonce ledger in `$OAK_RUNNER_HOME` is corrupt; it deliberately refuses rather than reading as empty	Inspect `consumed-nonces.json`; remove it only if you accept that previously burned nonces become replayable

`OAK-WORKSPACE-CORRUPT` on every command	Manifest unreadable, or written by a format this build does not know	Restore from your last `oak export` — this is why the manual says export before upgrading
`run-once` exited 0 but printed `denied <dispatch>: <reason>`	Denying a dispatch is the runner working correctly; exit status reports whether the runner ran, not whether anything succeeded	Read the signed messages (`oak ingest`) or the journal, and look the denial code up in the chapter 6 table or `docs/error-codes.md`
Runner exits 64 or 70 with a code on stderr	The runner itself could not run: 64 is a missing required environment variable, 70 a configuration or verification refusal that stopped it before processing (e.g. `OAK-RUNNER-CONFIG`)	Fix the environment or target profile it names — every `OAK-*` code is documented with its raise site in `docs/error-codes.md`
`oak approve apply` refuses with `OAK-APPROVAL-ARCHITECTURE`	An install needs a current approval of the architecture first, and there is none, or it is not current for this plan	Run `oak approve architecture`, then `oak approve apply` again

The full runbook is `docs/operations.md`; the complete generated error-code reference is `docs/error-codes.md`.

CHAPTER 9

Uninstalling completely

```
# 1 · Stack, volumes (your data), OAK-built images, fixture containers
docker compose down --volumes --remove-orphans
docker image rm -f $(docker image ls -q 'oak-community-*') 2>/dev/null || true
docker rm -f $(docker ps -aq --filter "label=oak.fixture=true") 2>/dev/null || true

# 2 · Home-directory state: PRIVATE KEYS, runner journals, consumed nonces
rm -rf ~/.oak

# 3 · Workspaces you created (you chose these paths; OAK does not track them),
#     and chapter 6's trust directory and mailboxes; trust/ holds DEVELOPMENT
PRIVATE KEYS
rm -rf ./workspace ./runner-demo ./runner-started ./case-export ./restored ./trust
./mailbox ./mailbox-started

# 4 · Build output, venv and caches inside the checkout
make clean-all

# 5 · Confirm — run this BEFORE step 6, which deletes the script itself
python scripts/check_clean_machine.py

# 6 · Finally the checkout, if you no longer want the source
cd .. && rm -rf OAK
```

`~/.oak` holds private signing keys, the runner's journal, consumed nonces and its revocation-sequence mark — export anything you want to keep first. It also holds any model-provider key stored in the file backend, under `~/.oak/credentials`, plus the model selection in `~/.oak/models`. A key stored in the operating-system keychain is *not* under `~/.oak` and survives this: remove those first, one family at a time, with `oak models remove-key <family>`. Chapter 6 placed its trust directory and two mailboxes in the repository root and its runner homes inside `runner-demo/` and `runner-started/`: `trust/` holds development private keys, which is why step 3 names it explicitly rather than leaving it to the checkout's removal. Shared caches (`~/.cache/uv`, the pnpm store, Playwright browsers) and the third-party images OAK pulled (`postgres:17.6-alpine`, `python:3.13.12-slim`, `node:24.18.0-alpine`, `nginxinc/nginx-unprivileged:1.29.1-alpine`, and `rancher/mirrored-pause`, which chapter 6's install pulled by digest) are shared Docker and toolchain state, yours to keep or remove with `docker image rm`. The full runbook version, including the named-volume and release-image variants, is `docs/operations.md` § Uninstall. Step 5 reports any OAK-related state that survived — run it while the checkout still exists, because step 6 removes the script along with everything else.

CHAPTER 10

Colophon: how this manual is built

The source of this manual is `docs/manual/manual.html`; the PDF is a rendering of it. Both are committed. To rebuild:

```
# screenshots, from a live journey against the Compose stack
docker compose up -d --build
OAK_MANUAL_SCREENS=1 pnpm --dir web exec playwright test e2e/manual-screens.spec.ts

# the PDF, using the same pinned Chromium the e2e suite uses
pnpm --dir web exec node ../docs/manual/build_manual.mjs
```

Every command in chapters 3, 5 and 6 was verified by running the full journeys, and the expected-output excerpts are what those runs printed. Chapter 6 was run against the Sprint 11 code that `0.9.0` ships unchanged, and chapter 5 against `0.8.0`; on `0.9.0` they print the same except the version. The repository's end-to-end suites (`tests/e2e/test_cli.py`, `tests/e2e/test_runner_journey.py`) exercise the same invocations continuously — except `oak revoke-approval`, which is covered by the revocation integration suite (`tests/integration/test_runner_revocation.py`) instead. Chapter 5's shell commands were run against the `0.8.0` Compose stack, as the tour records. The operations behind them are exercised by the interface conformance and MCP suites (`tests/integration/test_interface_conformance.py`, `tests/e2e/test_mcp_interface.py`), which need a PostgreSQL database and so run locally rather than in CI (`RR-019`). The screenshots were re-captured from the `0.9.0` stack. The diagrams are inline SVG in the source. This manual describes OAK Community 0.9.0 — a local-first developer release with no production or customer readiness claim; where this document and `docs/security/residual-risk.md` could ever disagree, the register wins.